Sistema de Processamento Paralelo de Requisições a um Banco de Dados em C

Natalie Krieger, Arthur Felipe Marques

Introdução

Este projeto consiste na implementação de um gerenciador de banco de dados simulado com processamento paralelo de requisições mútuas e suporte a uma linguagem SQL simplificada. É uma aplicação cliente-servidor executada em linha de comando onde múltiplos processos clientes enviam requisições ao processo servidor que faz amplo uso de *threads* e outros conceitos de programação paralela/concorrente para gerenciar as várias leituras e escritas que podem ocorrer ao longo da execução do programa.

Contexto da aplicação

Visto a importância e popularidade da programação paralela nos dias de hoje, se prova essencial que uma aplicação robusta faça o uso de múltiplos processos ou *threads* para tirar melhor proveito dos vários núcleos disponibilizados pelos processadores modernos, garantindo melhor usabilidade e experiência ao usuário final. Este projeto serviu para exercitar o conceito de programação paralela e comunicação entre processos aplicado em cenário real simulado: um sistema gerenciador de banco de dados.

Enunciado do trabalho

Se aprofundando na proposta do trabalho, ele se trata de desenvolver um sistema que simula o funcionamento interno de um gerenciador de requisições a um banco de dados, com múltiplos processos e threads, contendo:

- Um processo cliente que envia requisições de busca, inserção, atualização e deleção

de dados via IPC (pipe ou memória compartilhada);

- Um processo servidor recebe as requisições e usa várias threads para processá-las em paralelo;
- As threads utilizam mutex ou semáforo para garantir acesso seguro a uma estrutura compartilhada que simula um banco de dados (como um vetor ou um arquivo).

Os componentes requisitados foram:

Processo Cliente

- Envia "requisições de banco" para o servidor. Exemplo de requisição: SELECT nome WHERE id=5 ou INSERT id=7 nome='João';
- A comunicação é feita via pipe ou memória compartilhada.

Processo Servidor (Gerenciador de Banco)

- Lê as requisições do pipe ou memória compartilhada;
- Cria um pool de threads para processar cada requisição;
- As threads acessam um "banco de dados simulado" (pode ser um vetor ou arquivo .txt/.json/.sqlite) e usam mutex/semáforo para garantir a integridade dos dados;
- As respostas podem ser salvas em um segundo canal IPC, ou em um arquivo de log.

Requisições Suportadas

- INSERT: Adiciona um novo registro (linha) no "banco";
- DELETE: Remove um registro;
- SELECT: Busca e retorna dados com base em critérios simples (Ex.: por ID);
- UPDATE: Modifica um campo de um registro.

Fluxo do Sistema entre Processos/Threads

Cliente envia requisição SQL > Pipe ou Memória Compartilhada > Servidor (com threads) > Tabela de Dados com Mutex/Semáforo > Respostas/Logs ou Arquivo com Resultados

Implementação do projeto

A linguagem utilizada foi C, com amplo uso da biblioteca *pthread.h* para trabalhar o conceito de *multithreading* e paralelismo especificamente em sistemas POSIX (como Linux e MacOS), contando inclusive com uma *thread pool* alimentada por uma fila de *works* (ou *jobs*) que representam as requisições dos clientes.

Código do servidor que cria uma *thread* nova para cada cliente que se conecta:

```
printf("Servidor Named Pipe ouvindo em %s...\n\n\n", SOCK_PATH);

    while (1) {
        memset(&remote, 0, sizeof(remote));
        len = sizeof(remote);

        // Fica esperando conexões de clientes
        newsockfd = accept(sockfd, (struct sockaddr *)&remote, &len);

        if (newsockfd < 0)
        {
            perror("Falha em aceitar coneccao");
            close(sockfd);
            return 1;
        }

        // Cria um socket para a nova conexão com o cliente
        socket_node *new_socket = insertSocket(sock_list, newsockfd);

        // Cria uma thread nova para o cliente
        pthread_t clientThread;
        pthread_create(&clientThread, NULL, readSocket, new_socket);
        pthread_detach(clientThread);

        printf("Cliente conectado em socket %d!\n\n", newsockfd);

    }
```

Código que escuta a fila de *workers* e alimenta a *thread pool*:

```
// Função que ficará sendo executada continuadamente em cada uma das threads
na pool até o programa finalizar
static void *tpool_worker(void *arg)
{
    tpool_t      *tm = arg;
    tpool_work_t *work;

    while (1) {
        pthread_mutex_lock(&(tm->work_mutex));

            printf("\e[32m(Thread pool) Thread disponivel tid=%p\n\n\e[0m",
pthread_self());

        while (tm->work_first == NULL && !tm->stop)
            pthread_cond_wait(&(tm->work_cond), &(tm->work_mutex));

        if (tm->stop)
            break;

        work = tpool_work_get(tm);
        tm->working_cnt++;
        pthread_mutex_unlock(&(tm->work_mutex));

        if (work != NULL) {
                        printf("\e[32m(Thread pool) Thread trabalhando...
tid=%p\n\n\e[0m", pthread_self());
            work->output = work->func(work->arg);
                    printf("\e[32m(Thread pool) Thread finalizou trabalho
tid=%p\n\n\e[0m", pthread_self());
        }

        pthread_mutex_lock(&(tm->work_mutex));
        work->done = true;
        pthread_cond_broadcast(&(tm->done_cond));
        tm->working_cnt--;
        if (!tm->stop && tm->working_cnt == 0 && tm->work_first == NULL) {
            pthread_cond_broadcast(&(tm->working_cond));
        }
        pthread_mutex_unlock(&(tm->work_mutex));
    }

    tm->thread_cnt--;
    pthread_cond_broadcast(&(tm->working_cond));
    pthread_mutex_unlock(&(tm->work_mutex));
    return NULL;
}
```

Para garantir a sincronia das diferentes threads, evitando condições de corrida em variáveis globais e arquivos, foi feito amplo uso de *mutexes* e sinais de condição (*pthread_cond_t*), todos recursos da biblioteca *pthread.h*.

Código do servidor que faz o controle de leituras/escritas simultâneas:

```
      pthread_mutex_lock(&(write_mutex)); // Tranca 'write_mutex' (todos os
comandos são barrados aqui se ele estiver trancado)

    if (cmd.type == CMD_SELECT) {
        // Leitura
         pthread_mutex_unlock(&write_mutex); // Libera 'write_mutex', caso o
comando for de leitura
        pthread_mutex_lock(&wr_control_mutex);
        read_cont++;
        pthread_mutex_unlock(&wr_control_mutex);

    } else {
        // Escrita
        // 'write_mutex' não é liberado aqui
        pthread_mutex_lock(&wr_control_mutex);
        while (read_cont > 0)
                pthread_cond_wait(&done_cond, &wr_control_mutex); // Libera
'wr_control_mutex' e fica esperando 'done_cond' ser sinalizado por
'pthread_cond_broadcast()'
        pthread_mutex_unlock(&wr_control_mutex);

    }

    sleep(2); // sleep opcional para simular operações demoradas

    db_response *response = db_execute_statement(data);

    if (cmd.type == CMD_SELECT) {
        // Leitura
        pthread_mutex_lock(&wr_control_mutex);
        read_cont--;
        pthread_mutex_unlock(&wr_control_mutex);

    } else {
        // Escrita
        pthread_mutex_unlock(&write_mutex); // Libera 'write_mutex'
    }

    pthread_cond_broadcast(&done_cond); // Sinaliza que uma operação de banco
terminou (serve para operações de escrita que estão esperando 'done_cond')
```

De forma a simular um banco de dados, o programa faz leituras e escritas em um único arquivo CSV análogo a uma tabela em um banco estrutural, onde os clientes fazem buscas e alterações através de uma linguagem SQL ultra-simplificada.

Como o sistema é executado em diferentes processos (servidor e clientes), foi preciso fazer a comunicação entre processos, onde o método escolhido foi através de *Unix sockets* (ou *sockets locais*).

Código do cliente que faz a conexão ao servidor através de um *socket*:

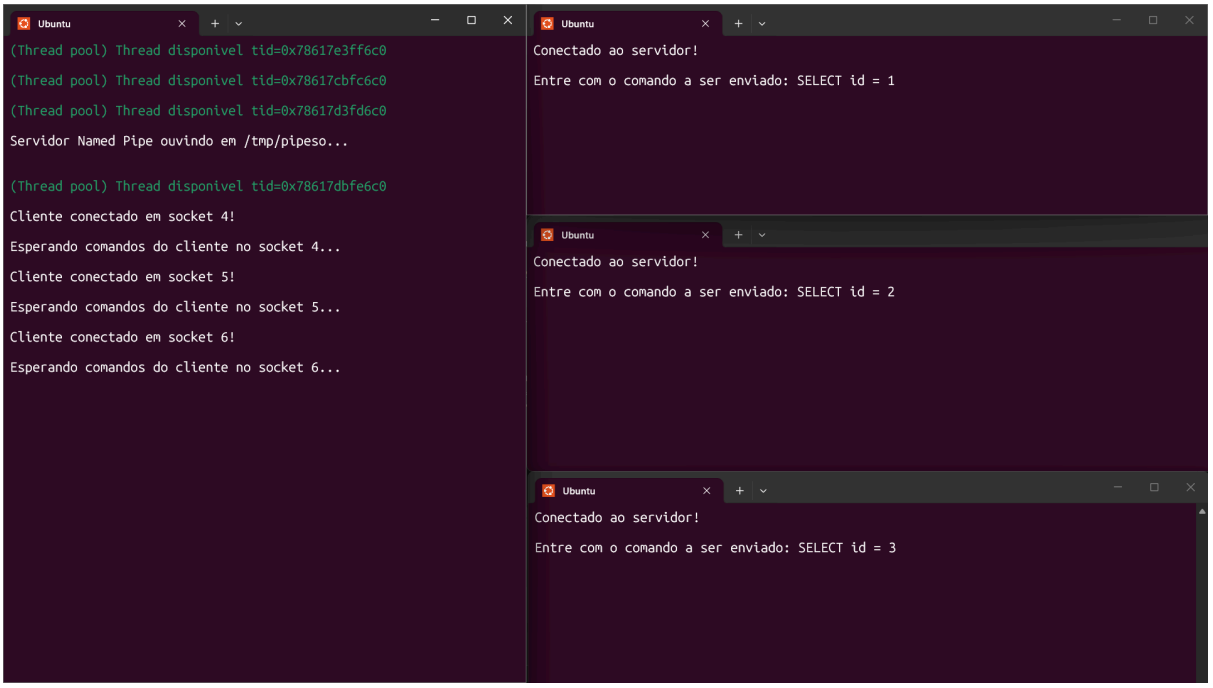
```
    int sockfd, len;
    struct sockaddr_un remote;
    char buffer[1024];

    // Create socket
    sockfd = socket(AF_UNIX, SOCK_STREAM, 0);
    if (sockfd < 0)
    {
        perror("Falha em criar o socket");
        return 1;
    }

    // Connect to server
    memset(&remote, 0, sizeof(remote));
    remote.sun_family = AF_UNIX;
    strncpy(remote.sun_path, SOCK_PATH, sizeof(remote.sun_path) - 1);
    len = strlen(remote.sun_path) + sizeof(remote.sun_family);
    if (connect(sockfd, (struct sockaddr *)&remote, len) < 0)
    {
        perror("Falha em conectar no servidor");
        close(sockfd);
        return 1;
    }

    printf("Conectado ao servidor!\n\n");
```

Resultados e análise

Testado com vários clientes enviando requisições simultâneas. Cada requisição é processada com delay deliberado de 2 segundos para simular processamentos pesados.

```
id,name,age
1,"Arthur Felipe",20
2,"Ronaldo",22
3,"Arthur, Silva",21
```

Estado inicial da tabela.



Servidor sendo executado na esquerda, clientes prestes a mandarem requisições de busca na direita.

```
Cliente conectado em socket 6!
Esperando comandos do cliente no socket 6...
Comando recebido: SELECT id = 1
(Thread pool) Thread trabalhando... tid=0x78617e3ff6c0
Comando recebido: SELECT id = 2
(Thread pool) Thread trabalhando... tid=0x78617cbfc6c0
Comando recebido: SELECT id = 3
(Thread pool) Thread trabalhando... tid=0x78617dbfe6c0
(Thread pool) Thread finalizou trabalho tid=0x78617e3ff6c0
(Thread pool) Thread disponivel tid=0x78617e3ff6c0
Resposta enviada para o cliente no socket 4.
Esperando comandos do cliente no socket 4...
(Thread pool) Thread finalizou trabalho tid=0x78617cbfc6c0
(Thread pool) Thread disponivel tid=0x78617cbfc6c0
Resposta enviada para o cliente no socket 5.
Esperando comandos do cliente no socket 5...
(Thread pool) Thread finalizou trabalho tid=0x78617dbfe6c0
(Thread pool) Thread disponivel tid=0x78617dbfe6c0
Resposta enviada para o cliente no socket 6.
Esperando comandos do cliente no socket 6...
```

```
Conectado ao servidor!
Entre com o comando a ser enviado: SELECT id = 1
Comando enviado ao servidor.
Esperando resposta...
Id: 1; Nome: Arthur Felipe; Idade: 20
Entre com o comando a ser enviado:
```

```
Conectado ao servidor!
Entre com o comando a ser enviado: SELECT id = 2
Comando enviado ao servidor.
Esperando resposta...
Id: 2; Nome: Ronaldo; Idade: 22
Entre com o comando a ser enviado:
```

```
Conectado ao servidor!
Entre com o comando a ser enviado: SELECT id = 3
Comando enviado ao servidor.
Esperando resposta...
Id: 3; Nome: Arthur, Silva; Idade: 21
Entre com o comando a ser enviado:
```

Respostas sendo entregues simultaneamente.

Como esperado, o servidor consegue processar as requisições de leitura em paralelo e entregar uma resposta satisfatória para todos os clientes simultaneamente.

Já requisições que demandam alteração na tabela precisam ser cuidadosamente gerenciadas para não executarem junto com outra operação de escrita ou leitura:

```
Cliente conectado em socket 6!
Esperando comandos do cliente no socket 6...
Comando recebido: SELECT id = 1
(Thread pool) Thread trabalhando... tid=0x78617e3ff6c0
Comando recebido: SELECT id = 2
(Thread pool) Thread trabalhando... tid=0x78617cbfc6c0
Comando recebido: SELECT id = 3
(Thread pool) Thread trabalhando... tid=0x78617dbfe6c0
(Thread pool) Thread finalizou trabalho tid=0x78617e3ff6c0
(Thread pool) Thread disponivel tid=0x78617e3ff6c0
Resposta enviada para o cliente no socket 4.
Esperando comandos do cliente no socket 4...
(Thread pool) Thread finalizou trabalho tid=0x78617cbfc6c0
(Thread pool) Thread disponivel tid=0x78617cbfc6c0
Resposta enviada para o cliente no socket 5.
Esperando comandos do cliente no socket 5...
(Thread pool) Thread finalizou trabalho tid=0x78617dbfe6c0
(Thread pool) Thread disponivel tid=0x78617dbfe6c0
Resposta enviada para o cliente no socket 6.
Esperando comandos do cliente no socket 6...
```

```
Conectado ao servidor!
Entre com o comando a ser enviado: SELECT id = 1
Comando enviado ao servidor.
Esperando resposta...
Id: 1; Nome: Arthur Felipe; Idade: 20
Entre com o comando a ser enviado: INSERT id = 4, name = "Emerson Rocha", age = 24
```

```
Conectado ao servidor!
Entre com o comando a ser enviado: SELECT id = 2
Comando enviado ao servidor.
Esperando resposta...
Id: 2; Nome: Ronaldo; Idade: 22
Entre com o comando a ser enviado: SELECT = 4
```

```
Conectado ao servidor!
Entre com o comando a ser enviado: SELECT id = 3
Comando enviado ao servidor.
Esperando resposta...
Id: 3; Nome: Arthur, Silva; Idade: 21
Entre com o comando a ser enviado: UPDATE id = 4, name = "Emerson Rocha Silva", age = 5
7
```

Requisições de inserção, busca e alteração, todas serão enviadas ao mesmo tempo.

```
Ubuntu
Resposta enviada para o cliente no socket 6.

Esperando comandos do cliente no socket 6...

Comando recebido: INSERT id = 4, name = "Emerson Rocha", age = 24

(Thread pool) Thread trabalhando... tid=0x78617e3ff6c0

Comando recebido: SELECT id = 4

(Thread pool) Thread trabalhando... tid=0x78617d3fd6c0

Comando recebido: UPDATE id = 4, name = "Emerson Rocha Silva", age
 = 57

(Thread pool) Thread trabalhando... tid=0x78617cbfc6c0

(Thread pool) Thread finalizou trabalho tid=0x78617e3ff6c0

(Thread pool) Thread disponivel tid=0x78617e3ff6c0

Resposta enviada para o cliente no socket 4.

Esperando comandos do cliente no socket 4...

(Thread pool) Thread finalizou trabalho tid=0x78617d3fd6c0

(Thread pool) Thread disponivel tid=0x78617d3fd6c0

Resposta enviada para o cliente no socket 5.

Esperando comandos do cliente no socket 5...

(Thread pool) Thread finalizou trabalho tid=0x78617cbfc6c0

(Thread pool) Thread disponivel tid=0x78617cbfc6c0

Resposta enviada para o cliente no socket 6.

Esperando comandos do cliente no socket 6...
```

```
Ubuntu
Id: 1; Nome: Arthur Felipe; Idade: 20

Entre com o comando a ser enviado: INSERT id = 4, name = "Emerson Rocha", age = 24

Comando enviado ao servidor.
Esperando resposta...

Insert: realizado com sucesso!

Entre com o comando a ser enviado:
```

```
Ubuntu
Comando enviado ao servidor.
Esperando resposta...

Id: 2; Nome: Ronaldo; Idade: 22

Entre com o comando a ser enviado: SELECT id = 4

Comando enviado ao servidor.
Esperando resposta...

Id: 4; Nome: Emerson Rocha; Idade: 24

Entre com o comando a ser enviado:
```

```
Ubuntu
Id: 3; Nome: Arthur, Silva; Idade: 21

Entre com o comando a ser enviado: UPDATE id = 4, name = "Emerson Rocha Silva", age = 5
7

Comando enviado ao servidor.
Esperando resposta...

Update: realizado com sucesso!

Entre com o comando a ser enviado:
```

Respostas sendo entregues sem erros.

```
id,name,age
1,"Arthur Felipe",20
2,"Ronaldo",22
3,"Arthur, Silva",21
4,"Emerson Rocha Silva",57
```

Estado final da tabela.

Como esperado, o servidor conseguiu realizar as operações com consistência e entregar as respostas para os clientes. No exemplo acima, o servidor processa o *INSERT* primeiro, fazendo com que as duas outras operações esperam a escrita terminar. Feito isso, a operação de *SELECT* é processada enquanto o *UPDATE* aguarda. Por fim, ao término do *SELECT*, a operação de *UPDATE* é processada, mantendo a consistência dos dados em todas as operações.

Conclusão

Através de técnicas de paralelismo, *multithreading* e sincronização conseguimos produzir uma aplicação robusta e adequada para o atual ecossistema computacional, fazendo uso proveitoso dos múltiplos núcleos e *threads* que nos são disponibilizados e ao mesmo tempo garantindo a consistência dos dados em qualquer que seja o fluxo de execução. Também foi utilizado *Unix sockets* como estratégia de comunicação entre processos, o que é interessante pois grande parte da API oferecida pela biblioteca de *sockets* pode ser aproveitada para implementar o uso de *network sockets*, que possibilitam a comunicação de diferentes máquinas pela internet, se aproximando ainda mais de um cenário realista.